

Universidad Americana

Facultad de Ingeniería y Arquitectura



Programación Orientada a Objetos II

MichiMaker - Documentación Técnica

Grupo #3:

- Dennis Amaru Cruz Abrego
- Richard Bernard Castro Fonseca

Profesor:

José Durán García

Managua, 24 de junio del 2026

Introducción

En los últimos años, las aplicaciones móviles han incorporado cada vez más funcionalidades relacionadas con el procesamiento de imágenes y la inteligencia artificial, permitiendo ofrecer experiencias interactivas y personalizadas a los usuarios. Dentro de este contexto surge MichiMaker, una aplicación móvil desarrollada para dispositivos Android que permite transformar fotografías humanas mediante filtros inspirados en distintos tipos de felinos (gato, tigre y león), combinando técnicas de visión artificial, procesamiento digital de imágenes y almacenamiento persistente de información.

El objetivo principal del proyecto es brindar una herramienta capaz de detectar automáticamente rostros en fotografías y superponer elementos gráficos que simulan características felinas, tales como orejas y hocicos. Además de generar la transformación visual, la aplicación permite que los usuarios registren una cuenta, almacenen sus creaciones en un servidor remoto y gestionen posteriormente su historial de transformaciones mediante operaciones de creación, consulta, actualización y eliminación de registros.

Para alcanzar estos objetivos se empleó una arquitectura cliente-servidor. La aplicación móvil fue desarrollada utilizando Kotlin y Jetpack Compose como tecnologías principales para la construcción de la interfaz de usuario. El procesamiento de rostros se implementó mediante Google ML Kit, mientras que la comunicación con el servidor se realizó utilizando Retrofit y servicios REST. Por su parte, el backend fue desarrollado con Spring Boot y utiliza PostgreSQL como sistema gestor de bases de datos para garantizar la persistencia de la información.

El desarrollo de MichiMaker permitió integrar conocimientos relacionados con programación móvil, diseño de interfaces, desarrollo de APIs REST, bases de datos relacionales y procesamiento de imágenes. Como resultado, se obtuvo una solución funcional capaz de generar transformaciones visuales, administrarlas mediante un sistema de cuentas de usuario y mantener la información sincronizada entre distintos dispositivos a través de una infraestructura cliente-servidor.

El presente documento describe la arquitectura, diseño, implementación y funcionamiento de la solución desarrollada, detallando los principales componentes del sistema y las decisiones técnicas adoptadas durante el proceso de construcción del proyecto.

Arquitectura General del Sistema

MichiMaker fue diseñado siguiendo una arquitectura cliente-servidor, en la cual la aplicación móvil Android y el servidor backend funcionan como componentes independientes que se comunican mediante servicios web REST. Esta separación permite distribuir las responsabilidades del sistema, facilitando el mantenimiento, la escalabilidad y la reutilización de los distintos módulos de software.

La capa cliente está compuesta por una aplicación móvil desarrollada en Kotlin utilizando Jetpack Compose para la construcción de las interfaces gráficas. Desde esta capa, el usuario puede registrarse, iniciar sesión, capturar o seleccionar imágenes, aplicar filtros inspirados en distintas razas de gatos y administrar el historial de transformaciones almacenadas en su cuenta. Además de gestionar la interacción con el usuario, la aplicación es responsable del procesamiento inicial de las imágenes mediante técnicas de visión artificial implementadas con Google ML Kit.

La lógica de presentación del cliente se organiza siguiendo el patrón MVVM (Model-View-ViewModel). Las pantallas representan la capa de vista, encargada de mostrar la información y capturar las acciones del usuario. Los ViewModels actúan como intermediarios entre la interfaz y la lógica de negocio, administrando el estado de la aplicación y coordinando las operaciones necesarias para cada funcionalidad. Finalmente, los repositorios encapsulan el acceso a las fuentes de datos externas, abstraen las llamadas a la API y permiten desacoplar la lógica de negocio de los detalles de comunicación con el servidor.

La capa de servicios está implementada mediante Retrofit, biblioteca que permite consumir servicios REST de forma sencilla y estructurada. A través de esta capa se realizan operaciones relacionadas con la autenticación de usuarios, almacenamiento de transformaciones, consulta del historial, actualización de leyendas y eliminación de registros.

Todas las solicitudes son enviadas mediante HTTP y los datos se intercambian utilizando objetos DTO serializados en formato JSON.

El backend fue desarrollado utilizando Spring Boot, framework que proporciona una infraestructura robusta para la creación de aplicaciones empresariales basadas en Java. Dentro del servidor se implementan controladores REST encargados de recibir las solicitudes provenientes del cliente, procesarlas y devolver las respuestas correspondientes. La lógica de acceso a datos se encuentra encapsulada en repositorios JPA, los cuales permiten interactuar con la base de datos mediante entidades persistentes y consultas especializadas.

La persistencia de la información se realiza mediante PostgreSQL, donde se almacenan los datos de los usuarios registrados y las transformaciones generadas por cada cuenta. Cada transformación conserva información como el filtro aplicado, la fecha de creación, la leyenda asociada y la imagen resultante codificada en formato binario. Gracias a este enfoque, las transformaciones permanecen disponibles incluso cuando la aplicación es desinstalada o utilizada desde un dispositivo diferente.

Desde una perspectiva funcional, el flujo principal del sistema inicia cuando un usuario captura o selecciona una imagen. Posteriormente, el módulo de procesamiento detecta el rostro presente en la fotografía y calcula la posición adecuada para superponer los elementos gráficos correspondientes al filtro seleccionado. Una vez generada la transformación, el usuario puede asignarle una leyenda y almacenarla en el servidor. Finalmente, las transformaciones guardadas pueden ser consultadas, modificadas o eliminadas desde el historial personal de la cuenta.

Esta arquitectura modular permitió separar claramente las responsabilidades de procesamiento de imágenes, presentación visual, comunicación de red y persistencia de datos, contribuyendo a la construcción de una solución mantenible y adaptable a futuras ampliaciones del proyecto.

Gestión de Usuarios

La gestión de usuarios constituye uno de los componentes fundamentales de MichiMaker, ya que permite asociar cada transformación realizada con una cuenta específica y garantizar que la información de cada usuario permanezca aislada del resto. El sistema implementa un esquema de autenticación basado en credenciales tradicionales, donde cada usuario debe registrarse proporcionando un nombre de usuario y una contraseña. Una vez completado el registro, la información es enviada al servidor mediante servicios REST desarrollados en Spring Boot, donde se valida y almacena de forma persistente en la base de datos PostgreSQL.

Durante el proceso de inicio de sesión, la aplicación verifica las credenciales ingresadas consultando la información almacenada en el servidor. Si la autenticación es exitosa, los datos básicos del usuario son recuperados y almacenados localmente mediante la clase 'SessionManager', la cual se encarga de mantener la sesión activa mientras la aplicación se encuentra en ejecución. Gracias a este mecanismo, el usuario no necesita volver a autenticarse cada vez que navega entre pantallas, ya que la información de la sesión puede ser consultada desde cualquier componente de la aplicación.

La entidad 'Usuario' representa el modelo principal de la gestión de usuarios dentro del backend. Esta entidad almacena atributos como el identificador único, nombre de usuario y contraseña. A partir de esta estructura, cada transformación registrada queda vinculada a un usuario específico mediante una relación entre entidades, permitiendo que el historial de transformaciones sea personalizado para cada cuenta registrada. De esta forma, dos usuarios distintos pueden utilizar la aplicación simultáneamente sin compartir información entre sí.

En la capa de acceso a datos, la persistencia de usuarios se encuentra delegada al repositorio 'UsuarioRepository', el cual utiliza Spring Data JPA para realizar operaciones de consulta y almacenamiento sobre la base de datos. Este repositorio permite recuperar usuarios por su identificador o por su nombre de usuario, facilitando tanto los procesos de autenticación como la obtención de información de perfil. Gracias al uso de JPA, la aplicación evita la necesidad de escribir consultas SQL complejas de forma manual, simplificando considerablemente el mantenimiento del código.

Desde la perspectiva del cliente Android, la información del usuario autenticado es utilizada en múltiples módulos del sistema. Por ejemplo, al guardar una transformación, el

identificador del usuario es enviado junto con los datos de la imagen para establecer la relación correspondiente en el servidor. Asimismo, pantallas como el historial, las estadísticas y el perfil utilizan los datos almacenados en la sesión para recuperar únicamente la información asociada al usuario actual. Esto garantiza que todas las consultas realizadas al backend sean personalizadas y coherentes con la cuenta autenticada.

Otro aspecto importante de la gestión de usuarios es el manejo del cierre de sesión. Cuando el usuario decide abandonar su cuenta, la información almacenada en `SessionManager` es eliminada, impidiendo que otras pantallas puedan acceder a datos de autenticación previamente guardados. Como resultado, cualquier intento de acceder a funcionalidades que requieren autenticación redirige automáticamente al usuario hacia la pantalla de inicio de sesión, reforzando la seguridad y la integridad de los datos.

En conjunto, el módulo de gestión de usuarios proporciona la base necesaria para personalizar la experiencia dentro de MichiMaker, permitiendo que cada transformación, estadística e historial se encuentre correctamente asociado a la cuenta que la generó. Esta estructura también facilita futuras ampliaciones del sistema, como la incorporación de recuperación de contraseñas, edición avanzada de perfiles o mecanismos de autenticación más robustos.

Procesamiento de Imágenes

El procesamiento de imágenes representa el núcleo funcional de MichiMaker, ya que es el componente encargado de transformar una fotografía convencional en una representación visual inspirada en distintas razas de gatos. Para lograr este objetivo, la aplicación combina técnicas de visión artificial, detección facial y manipulación gráfica, permitiendo que los elementos del filtro se adapten automáticamente a las características del rostro presente en la imagen.

El proceso comienza cuando el usuario captura una fotografía utilizando la cámara del dispositivo o selecciona una imagen existente desde la galería. Una vez obtenida la imagen, esta es convertida a un formato compatible con las bibliotecas de procesamiento utilizadas por la aplicación. Posteriormente, la fotografía es enviada al módulo de detección facial,

implementado mediante Google ML Kit, una herramienta especializada en tareas de visión artificial para dispositivos móviles.

La detección facial es gestionada por la clase `FaceDetectorManager`, cuya responsabilidad principal consiste en identificar la presencia de rostros dentro de la imagen y obtener información detallada sobre su posición y orientación. Entre los datos recuperados se encuentran las coordenadas del rostro, las dimensiones de su área de detección y los ángulos de inclinación de la cabeza. Esta información es fundamental para garantizar que los elementos gráficos del filtro se ubiquen correctamente sobre la fotografía.

Una vez detectado el rostro principal, el sistema selecciona el filtro elegido por el usuario. Cada filtro se encuentra definido dentro del catálogo de filtros (`FilterCatalog`), donde se especifican los recursos gráficos asociados, como las imágenes de orejas y hocicos, así como los factores de escala necesarios para adaptarlos a diferentes tamaños de rostro. Esta estructura permite incorporar nuevos filtros de manera sencilla sin modificar la lógica principal de procesamiento.

La aplicación utiliza la clase `CatFilterManager` para coordinar todo el proceso de transformación. A partir de los datos obtenidos por el detector facial, este componente calcula las dimensiones y posiciones adecuadas para cada elemento gráfico. Las orejas son ubicadas sobre la parte superior del rostro, mientras que el hocico se posiciona en la región central de la cara. Además, el sistema toma en consideración la inclinación de la cabeza mediante el valor del ángulo de rotación detectado por ML Kit, permitiendo que los elementos gráficos roten de forma natural junto con el rostro.

La superposición de imágenes es realizada mediante la clase `OverlayManager`, encargada de dibujar los recursos gráficos sobre la fotografía original. Durante este proceso se aplican transformaciones geométricas que incluyen escalado, rotación y posicionamiento dinámico. Gracias a estas operaciones, los filtros mantienen una apariencia coherente independientemente de las dimensiones de la fotografía o de la orientación de la persona retratada.

Con el fin de optimizar el rendimiento de la aplicación, los recursos gráficos utilizados por los filtros son almacenados temporalmente en memoria mediante un sistema de caché de

bitmaps. De esta manera, cuando un usuario aplica repetidamente el mismo filtro, la aplicación evita cargar nuevamente las imágenes desde los recursos del dispositivo, reduciendo el tiempo de procesamiento y el consumo de recursos.

Una vez completada la composición gráfica, la imagen resultante es almacenada temporalmente dentro del dispositivo utilizando las utilidades de almacenamiento implementadas en ImageStorageManager e ImageUtils. Posteriormente, la imagen puede ser visualizada en la pantalla de resultados, donde el usuario tiene la posibilidad de agregar una leyenda descriptiva antes de guardarla de forma permanente.

Finalmente, cuando el usuario decide almacenar la transformación, la imagen procesada es convertida a formato Base64 para facilitar su transmisión mediante servicios web REST. Este formato permite enviar la imagen como parte de la estructura JSON utilizada por la API, simplificando la comunicación entre la aplicación móvil y el servidor. Una vez recibida por el backend, la imagen es decodificada y almacenada en la base de datos junto con el resto de la información asociada a la transformación.

Gracias a la integración de detección facial automática, superposición dinámica de recursos gráficos y almacenamiento persistente, el módulo de procesamiento de imágenes constituye el elemento más innovador de MichiMaker, permitiendo generar transformaciones visuales de manera rápida, automática y personalizada para cada usuario.

Historial de Transformaciones

El módulo de Historial de Transformaciones representa uno de los componentes principales de la aplicación MichiMaker, ya que permite al usuario consultar y administrar los resultados generados previamente dentro de su cuenta. Su objetivo principal es proporcionar una vista personalizada donde cada usuario pueda acceder únicamente a las transformaciones que le pertenecen, manteniendo la información organizada y sincronizada con los datos almacenados en el servidor.

La implementación de esta funcionalidad se desarrolla bajo el patrón de arquitectura utilizado en la aplicación, separando la interfaz gráfica, la lógica de negocio y el acceso a datos. La

pantalla encargada de representar este módulo es `'HistoryScreen'`, construida utilizando Jetpack Compose. Esta pantalla obtiene la información del usuario autenticado mediante el componente `'SessionManager'`, lo que permite identificar la cuenta activa y utilizar su identificador para solicitar únicamente las transformaciones asociadas a dicho usuario.

Al iniciar la pantalla, se crea una instancia de `'TransformacionViewModel'`, la cual recibe como dependencia un `'TransformacionRepository'` y el identificador del usuario. Esta separación permite que la interfaz no tenga responsabilidad directa sobre las consultas HTTP ni sobre la manipulación de datos, sino que únicamente observe los cambios del estado proporcionado por el ViewModel mediante un flujo reactivo basado en `'StateFlow'`.

El estado visual del historial se controla mediante la clase `'TransformacionUiState'`, la cual almacena información relacionada con el proceso de carga, la lista de transformaciones recuperadas, la cantidad total de registros encontrados y posibles errores ocurridos durante la comunicación con el servidor. Gracias a este mecanismo, la pantalla puede modificar su contenido dependiendo de la situación actual del sistema, mostrando indicadores de carga mientras se obtienen los datos, mensajes informativos cuando no existen registros y avisos de error cuando ocurre algún problema.

La recuperación de información se realiza desde el método `'cargarHistorial()'` dentro de `'TransformacionViewModel'`. Este método ejecuta una operación asíncrona utilizando `'viewModelScope'`, evitando bloquear la interfaz mientras se realiza la petición. Posteriormente, solicita al repositorio la lista de transformaciones asociadas al usuario mediante el método `'obtenerPorUsuario()'`, el cual realiza una llamada HTTP hacia la API REST desarrollada en Spring Boot. Una vez recibida la respuesta, el ViewModel actualiza el estado interno y provoca automáticamente que Compose redibuje la interfaz con la información obtenida.

La comunicación con el servidor se encuentra encapsulada dentro de `'TransformacionRepository'`. Esta capa funciona como intermediario entre la aplicación móvil y el servicio remoto, centralizando las operaciones relacionadas con las transformaciones. Para la consulta del historial se utiliza el endpoint definido mediante Retrofit en `'TransformacionApi'`, enviando el identificador del usuario como parámetro de la solicitud. La respuesta obtenida corresponde a una colección de objetos

``TransformacionFeedDto``, los cuales contienen no solamente la información de la transformación, sino también datos complementarios del usuario como su nombre.

Cada transformación recuperada es presentada dentro de la interfaz mediante componentes visuales tipo tarjeta utilizando ``Card`` de Material 3. Dentro de cada tarjeta se muestra el nombre del filtro utilizado, la fecha de creación, la imagen generada y la leyenda asociada. La lista se representa mediante ``LazyColumn``, un componente optimizado para mostrar colecciones grandes, ya que solamente renderiza los elementos visibles en pantalla y mejora el rendimiento del desplazamiento.

Un aspecto importante de esta implementación es el manejo del almacenamiento de imágenes. Debido a que las imágenes procesadas se almacenan en la base de datos como una representación en Base64, la aplicación debe convertir nuevamente esa información en una imagen visible para el usuario. Para esto, la pantalla utiliza una función auxiliar denominada ``bitmapDesdeBase64``, la cual decodifica la cadena recibida, convierte los bytes obtenidos en un objeto ``Bitmap`` y posteriormente permite mostrarlo mediante los componentes de imagen de Jetpack Compose.

La carga de imágenes también utiliza el mecanismo ``remember`` de Compose junto con identificadores de transformación e información de la imagen. Esto evita realizar conversiones innecesarias durante las recomposiciones de la interfaz, optimizando el consumo de recursos y mejorando la experiencia del usuario al navegar por el historial.

Además de la consulta de información, el historial incorpora operaciones de mantenimiento sobre los registros existentes. La aplicación permite modificar la leyenda asociada a una transformación mediante un modo de edición integrado dentro de cada tarjeta. Cuando el usuario selecciona la opción de editar, la aplicación cambia dinámicamente la interfaz mostrando un campo de texto donde se puede modificar el contenido. Al confirmar el cambio, el `ViewModel` ejecuta el método ``actualizarLeyenda()``, el cual envía la nueva información al servidor mediante una petición PUT.

Después de completar la actualización, el `ViewModel` modifica directamente el estado local mediante una operación sobre la lista existente. Esto permite reflejar el cambio inmediatamente en pantalla sin necesidad de volver a descargar todo el historial desde el

servidor. Esta estrategia mejora la velocidad de respuesta y mantiene una experiencia más fluida, aprovechando el modelo reactivo de Jetpack Compose.

De manera similar, la eliminación de transformaciones se realiza mediante el método `eliminarTransformacion()`. Esta operación envía una solicitud DELETE al backend indicando el identificador del registro que debe eliminarse. Si la operación es exitosa, el ViewModel actualiza la lista almacenada en memoria removiendo el elemento eliminado y reduciendo el contador total de transformaciones. Gracias a esto, el usuario observa el cambio inmediatamente sin requerir una recarga manual de la pantalla.

La capa de red utiliza Retrofit como cliente HTTP, configurado mediante `ApiClient`, donde se establece la URL base del servidor desplegado y el conversor JSON utilizado para transformar automáticamente las respuestas del backend en objetos Kotlin. La interfaz `TransformacionApi` define las rutas disponibles para crear, consultar, actualizar y eliminar transformaciones siguiendo los principios de una API REST.

En conjunto, el módulo de Historial de Transformaciones implementa un flujo completo de gestión de datos donde el usuario puede consultar, modificar y eliminar información persistente. La integración entre Jetpack Compose, ViewModel, StateFlow, Repository y Retrofit permite mantener una arquitectura organizada, escalable y con una separación clara de responsabilidades entre la interfaz, la lógica de aplicación y la comunicación con el servidor.

Con estos archivos de `StatsScreen`, `StatsScreenViewModel` y `StatsState` se puede documentar la sección de Estadísticas siguiendo el mismo estilo técnico del Historial. La idea es explicar que no es simplemente una pantalla con números, sino un módulo que procesa información existente y genera una vista analítica.

Estadísticas

El módulo de Estadísticas de MichiMaker tiene como propósito proporcionar al usuario una vista resumida sobre el uso que ha realizado de la aplicación. A diferencia del historial, cuyo

objetivo principal es mostrar los registros individuales de transformaciones, esta sección se enfoca en analizar dichos registros y convertirlos en información más comprensible, permitiendo observar la cantidad total de transformaciones creadas y la distribución de uso de los diferentes filtros disponibles dentro de la aplicación.

La pantalla encargada de presentar esta información es `'StatsScreen'`, desarrollada utilizando Jetpack Compose. Al igual que otros módulos principales de la aplicación, esta pantalla mantiene la separación de responsabilidades establecida mediante una arquitectura basada en capas. La interfaz gráfica se limita a representar la información recibida, mientras que la obtención y procesamiento de los datos se delega al componente `'StatsScreenViewModel'`.

Antes de cargar la información estadística, la pantalla verifica la existencia de un usuario autenticado mediante `'SessionManager'`. Esta validación permite garantizar que los datos mostrados correspondan únicamente a la cuenta actualmente activa. En caso de que no exista una sesión válida, la aplicación redirige al usuario hacia la pantalla de inicio de sesión, evitando que se realicen consultas sin un identificador de usuario asociado.

Una vez validada la sesión, la pantalla crea una instancia del repositorio encargado de manejar las transformaciones y la conecta con el `'StatsScreenViewModel'`, enviándole el identificador del usuario. Este identificador es utilizado posteriormente para consultar únicamente los registros pertenecientes a esa cuenta, manteniendo la privacidad y separación de información entre usuarios.

La lógica principal del módulo se encuentra dentro de `'StatsScreenViewModel'`. Al inicializarse, el ViewModel ejecuta automáticamente el método `'cargarEstadisticas()'`, el cual realiza una operación asíncrona mediante `'viewModelScope'`. Esto permite consultar la información desde el servidor sin bloquear la ejecución de la aplicación ni afectar la experiencia del usuario mientras espera la respuesta.

Para obtener los datos necesarios, el ViewModel reutiliza el método `'obtenerPorUsuario()'` disponible en `'TransformacionRepository'`. Esta decisión evita duplicar lógica de comunicación con el servidor, aprovechando la misma capa utilizada por otros módulos como el historial de transformaciones. El repositorio se encarga de realizar la solicitud HTTP

mediante Retrofit y convertir la respuesta recibida en objetos Kotlin que puedan ser procesados dentro de la aplicación.

Una vez obtenida la lista de transformaciones del usuario, el ViewModel realiza un procesamiento interno de la información. Primero calcula el total de registros utilizando el tamaño de la colección recibida. Posteriormente genera una agrupación basada en el nombre del filtro aplicado mediante la operación `groupBy`, obteniendo un conteo independiente para cada tipo de transformación existente.

Este procesamiento permite transformar una colección de datos individuales en información estadística. Por ejemplo, si un usuario posee múltiples transformaciones realizadas con distintos filtros, el sistema puede identificar cuántas veces fue utilizado cada uno y presentar esa distribución de manera resumida. De esta forma, la aplicación no solamente almacena información histórica, sino que también genera conocimiento sobre los patrones de uso del usuario.

El estado de esta pantalla es administrado mediante la clase `StatsState`, la cual funciona como un contenedor de información observable por Compose. Esta clase almacena el número total de transformaciones y un mapa donde cada clave representa un filtro utilizado y cada valor corresponde a la cantidad de veces que dicho filtro aparece registrado. Al trabajar con un estado inmutable, cualquier modificación realizada por el ViewModel provoca una actualización automática de la interfaz.

Dentro de la interfaz visual, la información se presenta mediante componentes de Material 3. El total de transformaciones se muestra dentro de una tarjeta (`Card`) diseñada para resaltar el indicador principal, mientras que la distribución por filtros se representa utilizando componentes individuales mediante `StatItem`. Esta estructura permite mantener una presentación clara y escalable, ya que nuevos tipos de estadísticas pueden incorporarse sin modificar completamente la pantalla.

El diseño visual utiliza un fondo degradado y una estructura organizada verticalmente mediante componentes `Column`, manteniendo la identidad visual de MichiMaker basada en colores suaves y una experiencia amigable. Aunque el módulo está orientado al análisis de

datos, conserva la misma línea gráfica del resto de la aplicación para ofrecer una experiencia consistente al usuario.

La actualización de información puede realizarse mediante el método ``recargar()``, el cual vuelve a ejecutar la carga de estadísticas. Esto permite que el módulo pueda mantenerse sincronizado cuando existan nuevos registros creados por el usuario, evitando mostrar información desactualizada.

En conjunto, el módulo de Estadísticas demuestra una capa adicional de valor sobre los datos almacenados en MichiMaker, transformando registros simples de transformaciones en información analítica. La integración entre Retrofit, Repository, ViewModel y Jetpack Compose permite obtener datos reales desde el servidor, procesarlos dentro de la aplicación y presentarlos de forma clara, manteniendo una arquitectura organizada y preparada para futuras ampliaciones.

API REST

La comunicación entre la aplicación móvil MichiMaker y el servidor se realiza mediante una API REST desarrollada utilizando Spring Boot. Esta API funciona como el punto de conexión entre el cliente Android y la base de datos PostgreSQL, permitiendo que la aplicación pueda almacenar, consultar, modificar y eliminar información sin tener acceso directo a la capa de persistencia.

La arquitectura implementada sigue un modelo cliente-servidor, donde la aplicación móvil actúa como consumidor de servicios y el backend se encarga de procesar las solicitudes, validar la información recibida y ejecutar las operaciones correspondientes sobre la base de datos. Esta separación permite mantener una estructura organizada, donde cada componente tiene una responsabilidad definida y facilita el mantenimiento del sistema.

El backend está construido utilizando controladores REST mediante las anotaciones proporcionadas por Spring Framework. Los controladores principales del sistema corresponden a ``TransformacionController`` y ``UsuarioController``, los cuales agrupan las

operaciones relacionadas con transformaciones de imágenes y gestión de usuarios respectivamente.

El controlador ``TransformacionController`` representa el núcleo de la comunicación relacionada con las imágenes generadas dentro de la aplicación. Este controlador está asociado a la ruta base ``/transformaciones``, desde donde se exponen los servicios necesarios para registrar nuevas transformaciones, consultar historiales, actualizar información y eliminar registros.

La creación de nuevas transformaciones se realiza mediante una solicitud HTTP POST utilizando el endpoint ``/transformaciones``. Cuando la aplicación móvil envía una nueva transformación, el servidor recibe un objeto ``TransformacionDto``, el cual contiene los datos necesarios como el nombre del filtro aplicado, la fecha, la imagen procesada, el identificador del usuario y la leyenda asociada.

Antes de almacenar la información, el controlador realiza una validación del usuario recibido. El identificador enviado desde la aplicación es utilizado para buscar el usuario correspondiente mediante ``UsuarioRepository``. Esta validación garantiza que cada transformación quede relacionada correctamente con una cuenta existente y evita almacenar registros sin propietario asociado.

Una característica importante del sistema es el manejo del almacenamiento de imágenes. Debido a que las transformaciones pueden ser imágenes generadas desde el dispositivo móvil, el backend recibe el contenido en formato Base64. El controlador procesa esta información eliminando encabezados adicionales si existen y posteriormente convierte la cadena en un arreglo de bytes mediante la clase ``Base64``. Finalmente, la imagen resultante es almacenada en la entidad ``Transformacion`` como un campo binario.

Este enfoque permite que la imagen quede almacenada directamente dentro de PostgreSQL como información persistente, eliminando la dependencia de archivos locales del dispositivo. De esta manera, una transformación puede ser recuperada posteriormente desde cualquier dispositivo donde el usuario inicie sesión, siempre que tenga acceso a su cuenta.

Para evitar exponer directamente las entidades internas de la base de datos, la API utiliza objetos de transferencia de datos o DTOs ('Data Transfer Objects'). Las clases 'TransformacionDto' y 'TransformacionFeedDto' funcionan como estructuras intermedias entre el cliente y el servidor. Esta estrategia permite controlar exactamente qué información es enviada y recibida, evitando problemas relacionados con la exposición de entidades JPA completas.

El DTO utilizado para operaciones de escritura contiene principalmente los datos necesarios para crear o modificar una transformación, mientras que 'TransformacionFeedDto' está diseñado para respuestas más completas. Este segundo objeto incorpora información adicional del usuario asociado, como el nombre de usuario, permitiendo que las respuestas puedan utilizarse directamente en módulos como el historial.

La API también implementa diferentes endpoints de consulta utilizando solicitudes GET. El endpoint '/transformaciones/feed' permite obtener todas las transformaciones registradas ordenadas por identificador descendente, facilitando la construcción de una vista general donde los registros más recientes aparecen primero.

De manera similar, el endpoint '/transformaciones/usuario/{id}' permite consultar únicamente las transformaciones pertenecientes a un usuario específico. Esta operación es utilizada principalmente por módulos como el historial y estadísticas, ya que garantiza que cada usuario visualice únicamente sus propios datos.

Las operaciones de actualización se implementan mediante solicitudes PUT. Actualmente, el sistema permite modificar la leyenda asociada a una transformación utilizando el endpoint '/transformaciones/{id}'. El servidor recibe la nueva información, localiza el registro existente mediante su identificador y actualiza únicamente el campo correspondiente antes de guardar los cambios nuevamente en la base de datos.

Las eliminaciones se realizan mediante solicitudes DELETE. La API dispone de dos mecanismos diferentes: eliminación individual de una transformación mediante su identificador y eliminación completa del historial asociado a un usuario. Esta última operación utiliza una consulta personalizada definida en 'TransformacionRepository', permitiendo eliminar todos los registros relacionados con una cuenta específica.

La comunicación con usuarios también se encuentra implementada mediante el controlador ``UsuarioController``. Este componente administra operaciones relacionadas con el registro, consulta y autenticación básica de usuarios. El registro se realiza mediante una solicitud POST hacia ``/usuarios``, donde los datos enviados son transformados desde ``UsuarioDto`` hacia una entidad persistente antes de almacenarse.

Para el inicio de sesión se utiliza el endpoint ``/usuarios/login``. La aplicación envía las credenciales del usuario y el servidor realiza la búsqueda mediante ``UsuarioRepository``. Si la información coincide, el backend devuelve los datos del usuario autenticado, permitiendo que la aplicación móvil almacene la sesión activa mediante su propio sistema de manejo de usuario.

La persistencia de datos se realiza utilizando Spring Data JPA mediante interfaces Repository. ``TransformacionRepository`` y ``UsuarioRepository`` extienden de ``JpaRepository``, lo cual proporciona automáticamente operaciones comunes como guardar, buscar, actualizar y eliminar registros. Además, se agregaron métodos personalizados para obtener transformaciones ordenadas y buscar por usuario.

La API se encuentra consumida desde la aplicación Android mediante Retrofit. La clase ``ApiClient`` configura la conexión hacia el servidor desplegado y genera las implementaciones necesarias de las interfaces de servicio. Dentro de ``TransformacionApi`` se encuentran definidos los métodos que representan los endpoints disponibles, asociando cada función Kotlin con su respectivo método HTTP mediante anotaciones como ``@GET``, ``@POST``, ``@PUT`` y ``@DELETE``.

En conjunto, la API REST de MichiMaker permite establecer una comunicación estructurada entre la aplicación móvil y el servidor. La implementación de controladores, DTOs, repositorios y entidades proporciona una arquitectura escalable donde las operaciones CRUD se encuentran claramente definidas. Esta estructura facilita futuras ampliaciones del sistema, como agregar nuevos tipos de filtros, mejorar la autenticación o incorporar nuevas funcionalidades sociales sobre las transformaciones generadas.

Perfecto. Para la sección de Base de Datos no hace falta que sea una descripción genérica de PostgreSQL, porque en tu caso hay una decisión importante: la persistencia está desplegada en la nube usando Render, y el backend también está desplegado ahí. Conviene explicar la relación entre ambos servicios y cómo se modelan los datos.

Con lo que me comentaste y los archivos anteriores, quedaría así:

Base de Datos

La persistencia de información de MichiMaker se implementa utilizando PostgreSQL como sistema gestor de base de datos relacional, desplegado mediante la plataforma en la nube Render. La base de datos se encuentra integrada con el servicio backend desarrollado en Spring Boot, también alojado en Render, permitiendo que la aplicación móvil pueda acceder a la información almacenada mediante solicitudes HTTP a través de la API REST.

La utilización de una base de datos remota permite que la información generada dentro de la aplicación no dependa únicamente del almacenamiento local del dispositivo. Esto garantiza que los datos del usuario permanezcan disponibles incluso si la aplicación es cerrada, reinstalada o ejecutada desde otro dispositivo, siempre que exista una sesión válida asociada a la cuenta correspondiente.

El modelo de datos de MichiMaker está compuesto principalmente por dos entidades: usuarios y transformaciones. La relación entre ambas entidades permite organizar la información de manera que cada usuario pueda mantener su propio historial independiente de imágenes procesadas.

La entidad principal del sistema corresponde a la tabla de usuarios, donde se almacenan los datos necesarios para identificar y administrar las cuentas dentro de la aplicación. Esta estructura contiene un identificador único generado automáticamente, un nombre de usuario, información personal básica como nombre y correo electrónico, una contraseña y una referencia opcional a la imagen de perfil del usuario.

Dentro del modelo implementado con JPA, la entidad `'Usuario'` mantiene una relación de uno a muchos con las transformaciones realizadas. Esto significa que un usuario puede tener múltiples registros asociados, mientras que cada transformación pertenece únicamente a una cuenta específica. Esta relación se representa mediante la clave foránea `'usuario_id'` dentro de la tabla de transformaciones.

La tabla de transformaciones representa el almacenamiento principal generado por la aplicación. Cada registro contiene información relacionada con una imagen procesada, incluyendo el filtro utilizado, la fecha de creación, la leyenda escrita por el usuario y la imagen resultante. Además, mantiene la referencia al usuario propietario mediante la relación establecida con la entidad `'Usuario'`.

Una decisión importante dentro del diseño de la base de datos fue almacenar las imágenes directamente dentro de PostgreSQL en formato binario. Para esto se utiliza una columna de tipo `'bytea'`, donde se guarda el contenido de la imagen después de ser convertido desde Base64 por el servidor. Este proceso permite que la aplicación pueda recuperar nuevamente la imagen completa cuando consulta los datos desde el backend.

Este enfoque evita depender de rutas locales del dispositivo o archivos externos que podrían perderse. Aunque inicialmente el sistema maneja una referencia mediante el campo `'rutaImagen'` para compatibilidad, la fuente principal de información visual corresponde al contenido binario almacenado dentro de la base de datos.

La conversión de imágenes ocurre durante el proceso de creación de una transformación. La aplicación móvil envía la imagen procesada en formato Base64 dentro del objeto `'TransformacionDto'`. Posteriormente, el backend recibe esta información, elimina encabezados adicionales si existen y convierte la cadena en bytes utilizando las herramientas estándar de Java. Finalmente, estos bytes son almacenados en la columna correspondiente dentro de PostgreSQL.

La comunicación con la base de datos no se realiza directamente desde la aplicación Android. En su lugar, el acceso se encuentra controlado por la capa backend mediante Spring Data JPA. Los repositorios definidos (`'UsuarioRepository'` y `'TransformacionRepository'`)

permiten ejecutar operaciones de consulta y modificación utilizando métodos preparados por JPA, reduciendo la necesidad de escribir consultas SQL manuales para operaciones comunes.

El repositorio de transformaciones incorpora métodos personalizados para obtener registros ordenados por fecha de creación, consultar las transformaciones pertenecientes a un usuario específico y eliminar todos los registros relacionados con una cuenta.

La estructura implementada mantiene una separación clara entre las capas del sistema. PostgreSQL funciona exclusivamente como almacenamiento persistente, Spring Boot actúa como intermediario encargado de procesar reglas y solicitudes, mientras que la aplicación móvil consume los datos mediante Retrofit. Esta organización evita que el cliente tenga acceso directo a la base de datos y mejora la seguridad y mantenibilidad del proyecto.

El uso de Render como plataforma de despliegue permite mantener todos los componentes principales del sistema funcionando en un entorno remoto. La aplicación Android se comunica con el servidor mediante la URL pública de la API, mientras que el backend establece la conexión interna con la instancia PostgreSQL alojada dentro del mismo proveedor. Esto simplifica la administración del proyecto al centralizar los servicios necesarios para la ejecución de la aplicación.

En conjunto, el diseño de base de datos implementado en MichiMaker permite almacenar usuarios, imágenes y metadatos de manera persistente y estructurada. La combinación de PostgreSQL, Spring Boot, JPA y Render proporciona una solución escalable donde la información puede ser consultada y modificada de forma segura mediante la arquitectura cliente-servidor establecida.

Conclusiones

El desarrollo de MichiMaker permitió construir una aplicación móvil completa que integra procesamiento de imágenes, almacenamiento persistente de información y una arquitectura cliente-servidor basada en tecnologías modernas. El proyecto logró combinar diferentes componentes tecnológicos como Jetpack Compose para la construcción de interfaces, herramientas de visión artificial para la aplicación de filtros sobre imágenes, Retrofit para la

comunicación con servicios externos, Spring Boot para la creación de la API REST y PostgreSQL como sistema de almacenamiento de datos.

La implementación de una arquitectura separada por capas permitió organizar el funcionamiento interno de la aplicación de manera más eficiente. La división entre pantallas, ViewModels, repositorios y servicios de red facilitó la distribución de responsabilidades, evitando que la interfaz gráfica tuviera contacto directo con la lógica de negocio o con la base de datos. Esta estructura permitió que funcionalidades como el historial, las estadísticas y la gestión de transformaciones pudieran desarrollarse de forma independiente y posteriormente integrarse dentro del flujo general de la aplicación.

Uno de los aspectos más relevantes del proyecto fue la incorporación de técnicas de procesamiento de imágenes mediante inteligencia artificial. La aplicación no se limita únicamente a almacenar fotografías, sino que permite modificar imágenes utilizando filtros basados en detección facial y superposición de elementos gráficos. Esto convirtió a MichiMaker en una solución interactiva donde el usuario puede generar contenido personalizado y conservar los resultados obtenidos.

La integración entre la aplicación móvil y el backend permitió implementar un sistema completo de persistencia de información. Gracias a la API REST desarrollada con Spring Boot, los usuarios pueden registrarse, iniciar sesión, guardar transformaciones, consultar historiales, actualizar leyendas y eliminar registros. La comunicación mediante solicitudes HTTP permitió mantener sincronizada la información entre el dispositivo móvil y el servidor remoto desplegado en Render.

El uso de PostgreSQL como base de datos permitió almacenar la información de manera estructurada, manteniendo relaciones entre usuarios y transformaciones realizadas. La decisión de guardar las imágenes procesadas como datos binarios dentro de la base de datos proporcionó una mayor independencia del almacenamiento local del dispositivo, asegurando que los resultados generados permanezcan disponibles para el usuario incluso al cambiar de dispositivo o reinstalar la aplicación.

La implementación del módulo de historial y estadísticas agregó una capa adicional de utilidad sobre los datos generados. El historial permite al usuario administrar sus creaciones

mediante operaciones de actualización y eliminación, mientras que el módulo estadístico transforma los registros almacenados en información resumida sobre el uso de filtros. Esto demuestra cómo los datos almacenados pueden aprovecharse para mejorar la experiencia del usuario más allá de una simple consulta de registros.

Durante el desarrollo también se aplicaron principios de diseño orientados a aplicaciones modernas, utilizando estados reactivos mediante `StateFlow`, componentes declarativos con Jetpack Compose y operaciones asíncronas mediante corrutinas. Estas decisiones permitieron construir una aplicación con interfaces dinámicas, capaces de responder a cambios en los datos sin necesidad de recargar manualmente las pantallas.

En conclusión, MichiMaker cumple con los objetivos planteados al ofrecer una aplicación móvil funcional que integra visión artificial, gestión de usuarios, almacenamiento en la nube y comunicación mediante servicios web. La solución desarrollada demuestra la integración de múltiples tecnologías dentro de un mismo sistema y establece una base preparada para futuras mejoras, como nuevos filtros de transformación, sistemas de interacción entre usuarios, autenticación más avanzada o ampliación de las capacidades de análisis de información.